[ADS5034-01] Computer Vision

10. Motion Tracking

Jae-Young Choi

Sungkyunkwan University

Outline

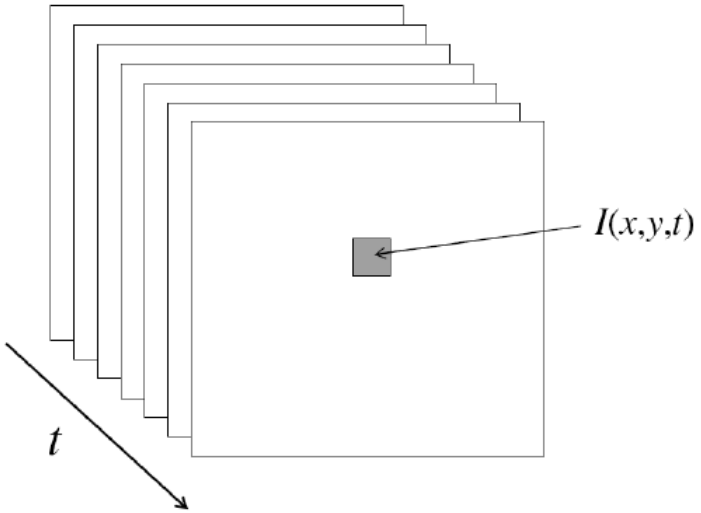


- Motion
- Optical Flow
  - Lucas-Kanade
  - Horn-Schunck
- Visual Tracking
  - Kanade-Lucas-Tomasi
  - Mean shift tracking
  - Cam shift tracking

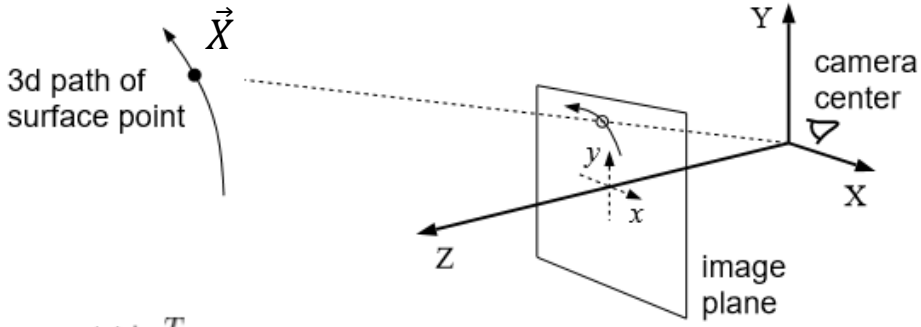
Motion

Motion

- From images to videos
  - A video is a sequence of frames captured over time
  - Now our image data is a function of space (x, y) and time (t)



Motion



- Introduction to Image Motion
  - A 3D point $\vec{X}$ follows a space-time path $\vec{X}(t)$
  - Its velocity is $\vec{V}$ is

$$\vec{\mathbf{V}} = \frac{d\vec{\mathbf{X}}(t)}{dt} = \left(\frac{dX(t)}{dt}, \frac{dY(t)}{dt}, \frac{dZ(t)}{dt}\right)^T$$

  - Perspective projection (for nodal length f) of the 3D path onto the
  - image plane produces a 2D path,

$$\vec{\mathbf{x}}(t) = (x(t), y(t)) = \left(\frac{fX(t)}{Z(t)}, \frac{fY(t)}{Z(t)}\right)$$

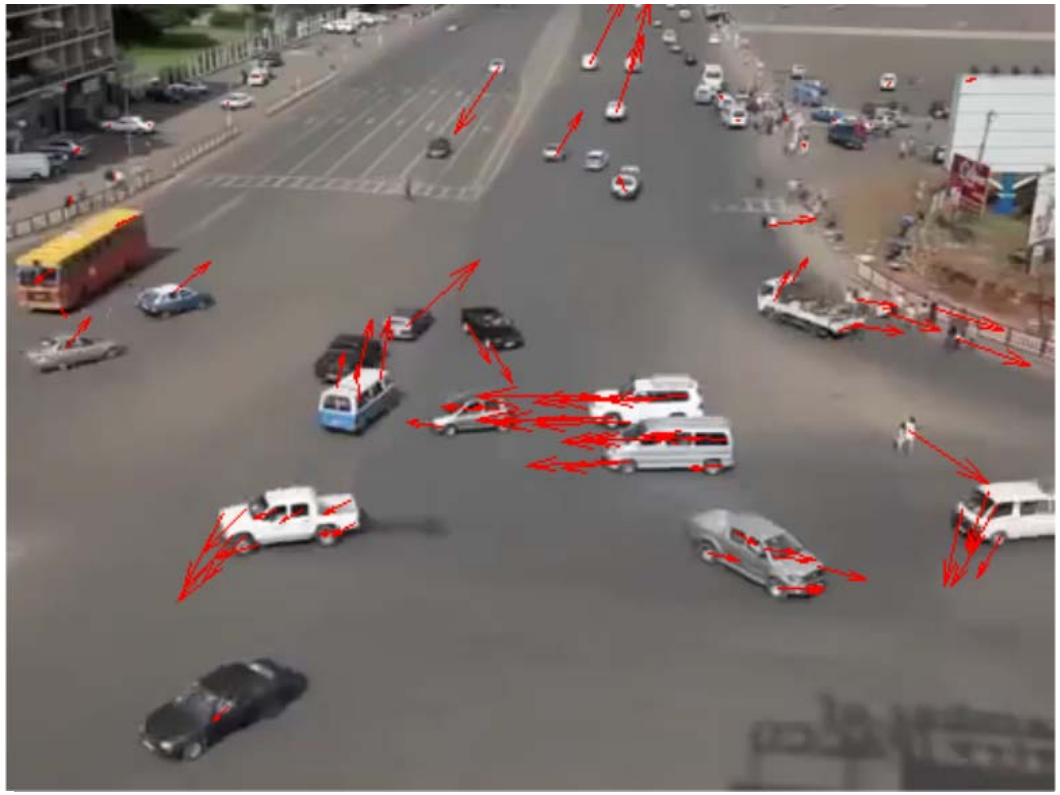
  - the instantaneous 2D velocity of which is

$$\vec{\mathbf{u}} = \left(\frac{dx(t)}{dt}, \frac{dy(t)}{dt}\right)^T$$

Motion

- Why is motion useful?
  - Motion is a rich source of information about the world

    - segmentation
    - surface structure from parallax
    - recognition
    - understanding behavior
    - understanding scene dynamic



Motion

- Types of motion
  - Moving with a fixed camera
  - Moving with a moving camera
  - Fixed object with a moving camera

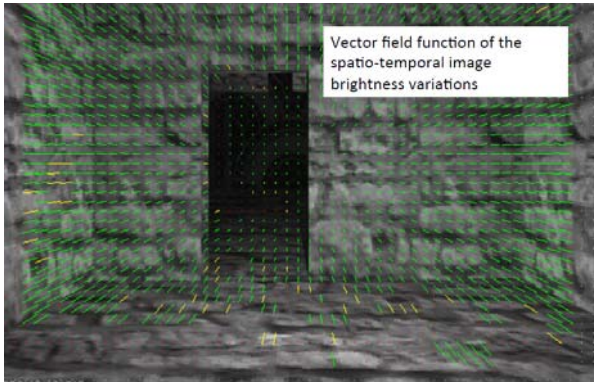






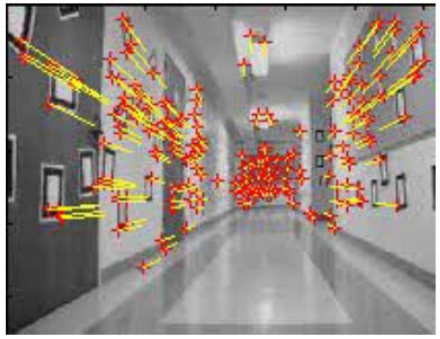
Motion

- Motion estimation techniques
  - Optical flow
    - Recover image motion at each pixel from spatio-temporal image brightness variations



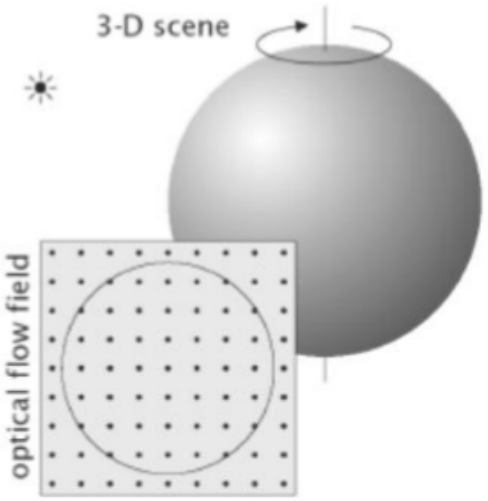
  - Feature-tracking
    - Extract visual features (corners, textured areas) and “track” them over multiple frames

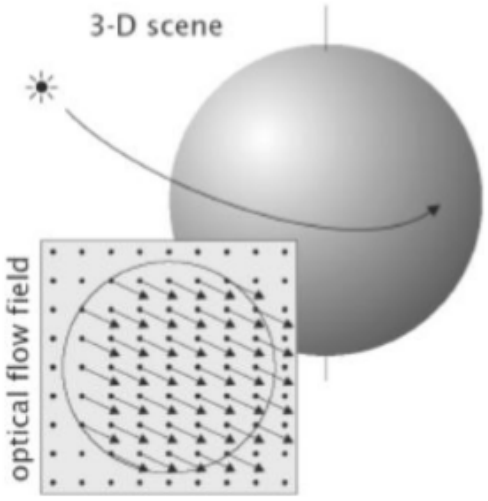




Motion

- Two Key Problems
  - Determine what image property to track
  - Determine how to track it

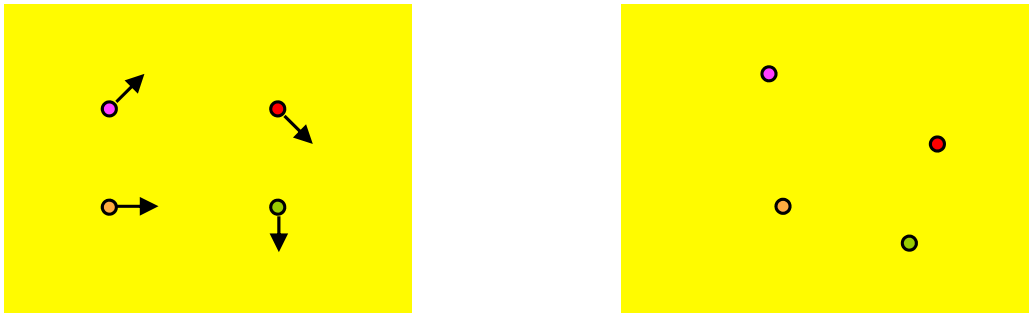




Optical Flow

Optical Flow

- Problem Definition
  - Given two consecutive image frames, estimate the motion of each pixel



$I(x, y, t)$ $I(x, y, t')$

Estimate the motion (flow) between
these two consecutive images

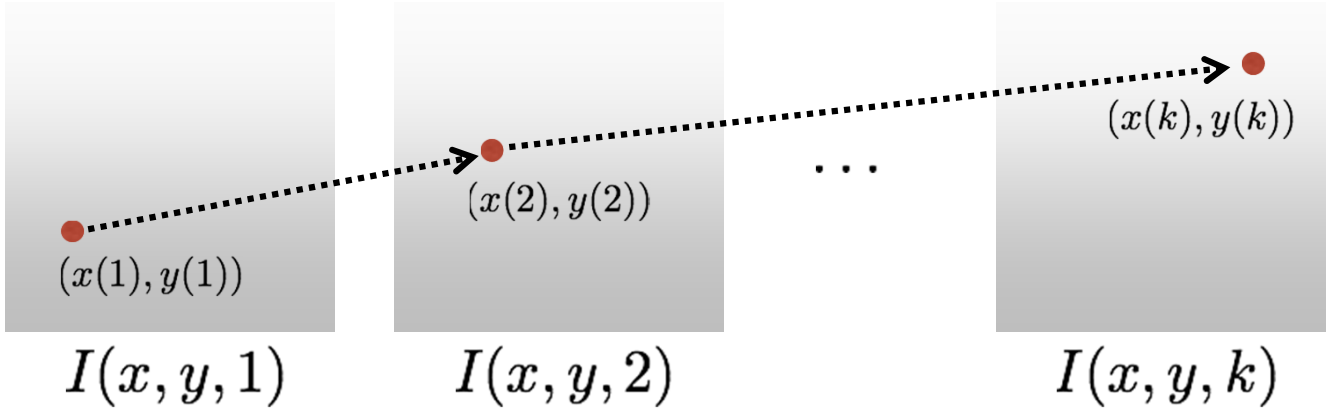
Optical Flow

- Assumptions
  - Color constancy
    - Brightness constancy for intensity images
    - Implication: allows for pixel to pixel comparison (not image features)
  - Small motion
    - pixels only move a little bit
    - Implication: linearization of the brightness constancy constraint

Optical Flow

- Assumptions 1
  - Brightness constancy

    Scene point moving through image sequence



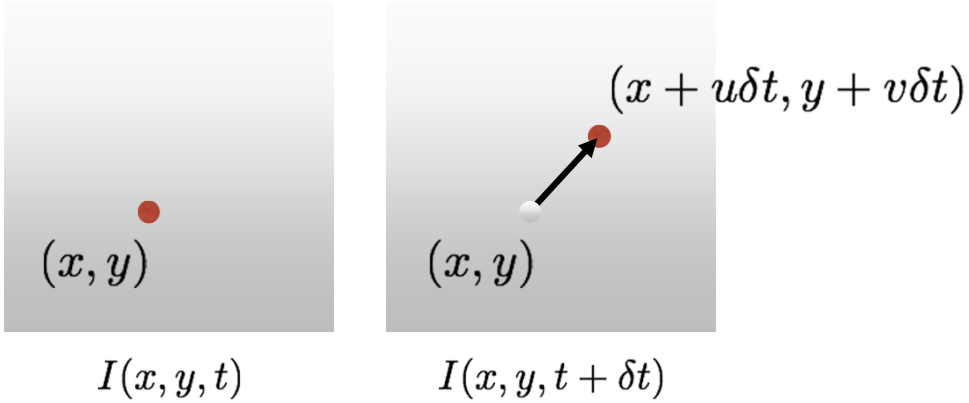
Assumption: Brightness of the point will remain the same

$$I(x(t), y(t), t) = C$$

constant

Optical Flow

- Assumptions 2
  - Small motion



Optical flow (velocities): (u, v) Displacement: $(\delta x, \delta y) = (u\delta t, v\delta t)$

For a *really small space-time step*…

$$I(x + u\delta t, y + v\delta t, t + \delta t) = I(x, y, t)$$

… the brightness between two consecutive image frames is the same

Optical Flow

- From the assumptions $I(x + u\delta t, y + v\delta t, t + \delta t) = I(x, y, t)$

Multivariable Taylor Series Expansion
(First order approximation, two variables)

$$f(x, y) \approx f(a, b) + f_x(a, b)(x - a) - f_y(a, b)(y - b)$$

$$I(x, y, t) + \frac{\partial I}{\partial x}\delta x + \frac{\partial I}{\partial y}\delta y + \frac{\partial I}{\partial t}\delta t = I(x, y, t)$$

assuming small motion

$$\frac{\partial I}{\partial x}\delta x + \frac{\partial I}{\partial y}\delta y + \frac{\partial I}{\partial t}\delta t = 0$$

divide by δt

take limit $\delta t \to 0$

$$\frac{\partial I}{\partial x}\frac{dx}{dt} + \frac{\partial I}{\partial y}\frac{dy}{dt} + \frac{\partial I}{\partial t} = 0$$

Brightness Constancy Equation

Optical Flow

- From the assumptions $I(x + u\delta t, y + v\delta t, t + \delta t) = I(x, y, t)$

$$\frac{\partial I}{\partial x}\frac{dx}{dt} + \frac{\partial I}{\partial y}\frac{dy}{dt} + \frac{\partial I}{\partial t} = 0$$

Brightness Constancy Equation

$$\underset{\text{(x-flow)}}{I_x u} + \underset{\text{(y-flow)}}{I_y v} + I_t = 0$$

shorthand notation

$$\underset{\text{(1 x 2)}}{\nabla I^{\top}} \underset{\text{(2 x 1)}}{\boldsymbol{v}} + I_t = 0$$

vector form

Optical Flow

- How to compute…

flow velocities

$$I_x u + I_y v + I_t = 0$$

Image gradients
(at a point p)

temporal gradient

$$I_x = \frac{\partial I}{\partial x} \quad I_y = \frac{\partial I}{\partial y}$$

spatial derivative

Forward difference
Sobel filter
…

$$u = \frac{dx}{dt} \quad v = \frac{dy}{dt}$$

optical flow

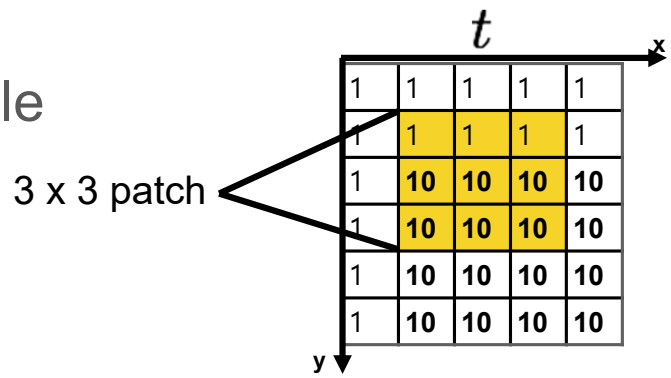
$$I_t = \frac{\partial I}{\partial t}$$

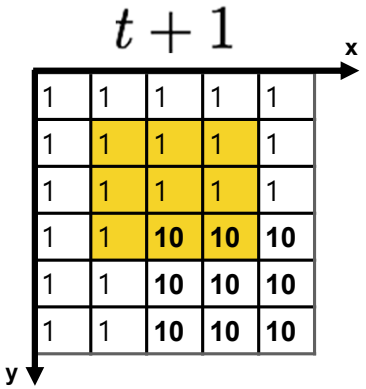
temporal derivative

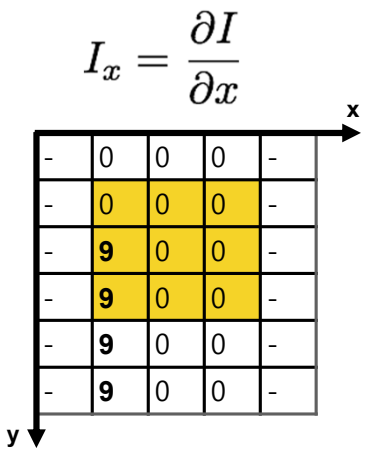
frame differencing

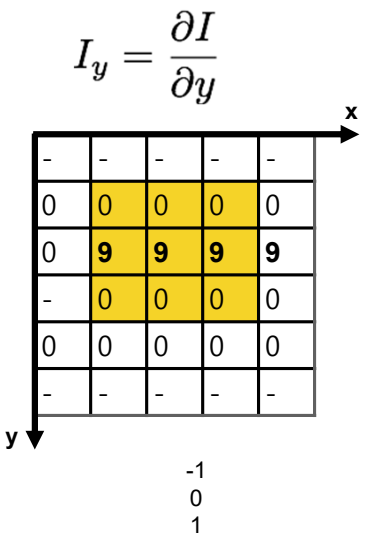
Optical Flow

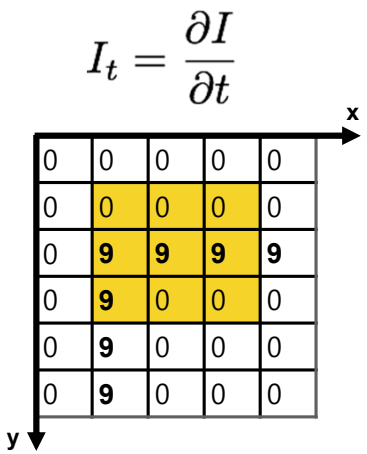
- Example











Optical Flow

- How to compute…

flow velocities

$$I_x u + I_y v + I_t = 0$$

Image gradients
(at a point p)

temporal gradient

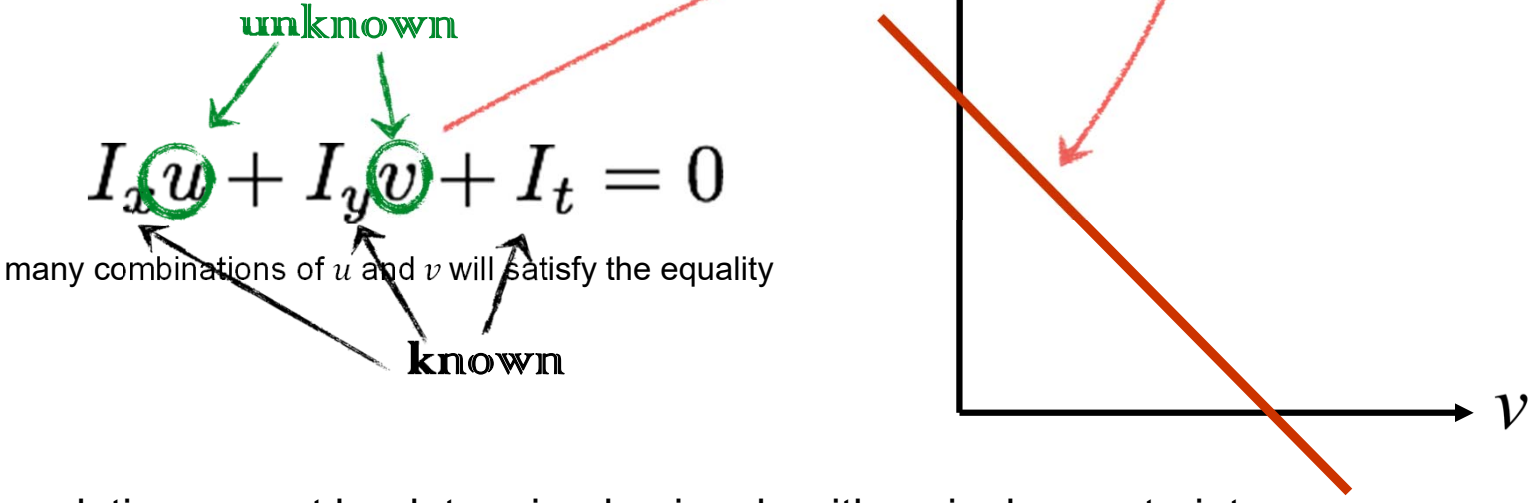
$I_x = \frac{\partial I}{\partial x} \quad I_y = \frac{\partial I}{\partial y}$	$u = \frac{dx}{dt} \quad v = \frac{dy}{dt}$	$I_t = \frac{\partial I}{\partial t}$
spatial derivative	**optical flow**	**temporal derivative**
Forward difference Sobel filter …	**We need to solve for this!** (this is the unknown in the optical flow problem)	frame differencing

→ (u, v) Solution lies on a line

Optical Flow

- How to compute…

Solution lies on a straight line



The solution cannot be determined uniquely with a single constraint
(a single pixel)

Optical Flow

- Where do we get more equations (constraints)?

$$\begin{bmatrix} I_x(\boldsymbol{p}_1) & I_y(\boldsymbol{p}_1) \\ I_x(\boldsymbol{p}_2) & I_y(\boldsymbol{p}_2) \\ \vdots & \vdots \\ I_x(\boldsymbol{p}_{25}) & I_y(\boldsymbol{p}_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\boldsymbol{p}_1) \\ I_t(\boldsymbol{p}_2) \\ \vdots \\ I_t(\boldsymbol{p}_{25}) \end{bmatrix}$$

Lucas-Kanade Optical Flow (1981)

method of differences

'constant' flow
(flow is constant for all pixels)

local method
(sparse)

$$\min_{\boldsymbol{u},\boldsymbol{v}} \sum_{ij} \left\{ E_d(i,j) + \lambda E_s(i,j) \right\}$$

Horn-Schunck Optical Flow (1981)

brightness constancy
small motion

'smooth' flow
(flow can vary from pixel to pixel)

global method
(dense)

Optical Flow (Lucas-Kanade)

- Lucas-Kanade Optical Flow

Where do we get more equations (constraints)?

$$I_x u + I_y v + I_t = 0$$

Assume that the surrounding patch (say 5x5) has
'**constant flow**'

- Flow is locally smooth
- Neighboring pixels have same displacement

Optical Flow (Lucas-Kanade)

- ‘constant’ flow
  - Using a 5 x 5 image patch, gives us 25 equations

$$
\begin{aligned}
I_x(\boldsymbol{p}_1)u + I_y(\boldsymbol{p}_1)v &= -I_t(\boldsymbol{p}_1) \\
I_x(\boldsymbol{p}_2)u + I_y(\boldsymbol{p}_2)v &= -I_t(\boldsymbol{p}_2) \\
&\vdots \\
I_x(\boldsymbol{p}_{25})u + I_y(\boldsymbol{p}_{25})v &= -I_t(\boldsymbol{p}_{25})
\end{aligned}
$$

$$
\underset{25 \times 2}{A} \qquad \underset{2 \times 1}{x} \qquad \underset{25 \times 1}{b}
$$

$$
\begin{bmatrix}
I_x(\boldsymbol{p}_1) & I_y(\boldsymbol{p}_1) \\
I_x(\boldsymbol{p}_2) & I_y(\boldsymbol{p}_2) \\
\vdots & \vdots \\
I_x(\boldsymbol{p}_{25}) & I_y(\boldsymbol{p}_{25})
\end{bmatrix}
\begin{bmatrix} u \\ v \end{bmatrix}
= -
\begin{bmatrix}
I_t(\boldsymbol{p}_1) \\
I_t(\boldsymbol{p}_2) \\
\vdots \\
I_t(\boldsymbol{p}_{25})
\end{bmatrix}
$$

Matrix form

Optical Flow (Lucas-Kanade)

- 'constant' flow

Least squares approximation

$$\hat{x} = \arg\min_x ||Ax - b||^2 \quad \text{is equivalent to solving} \quad A^\top A\hat{x} = A^\top b$$

To obtain the least squares solution solve:

$$\underbrace{\begin{bmatrix} \sum_{p\in P} I_x I_x & \sum_{p\in P} I_x I_y \\ \sum_{p\in P} I_y I_x & \sum_{p\in P} I_y I_y \end{bmatrix}}_{A^\top A} \underbrace{\begin{bmatrix} u \\ v \end{bmatrix}}_{\hat{x}} = -\underbrace{\begin{bmatrix} \sum_{p\in P} I_x I_t \\ \sum_{p\in P} I_y I_t \end{bmatrix}}_{A^\top b}$$

where the summation is over each pixel ***p*** in patch ***P***

$$x = (A^\top A)^{-1} A^\top b$$

Optical Flow (Lucas-Kanade)

- Barber’s pole illusion

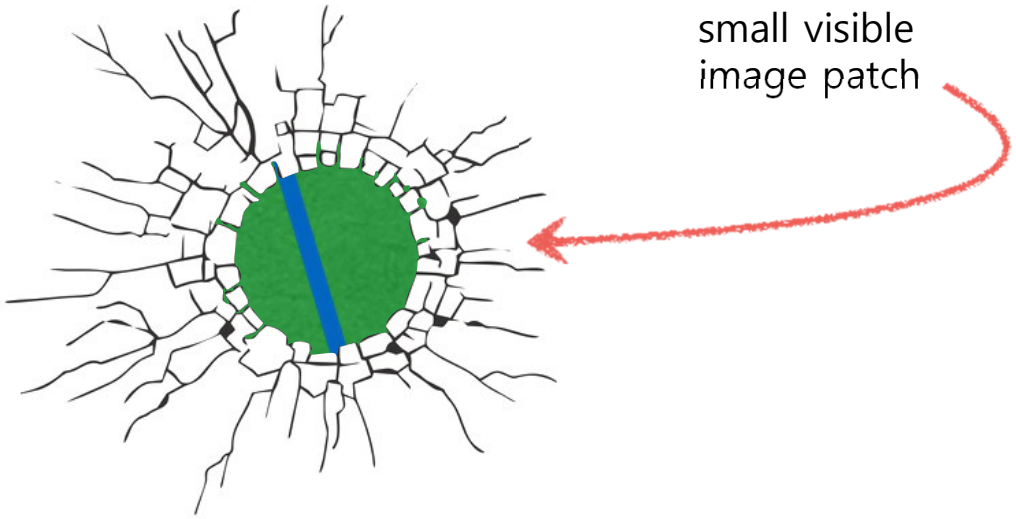






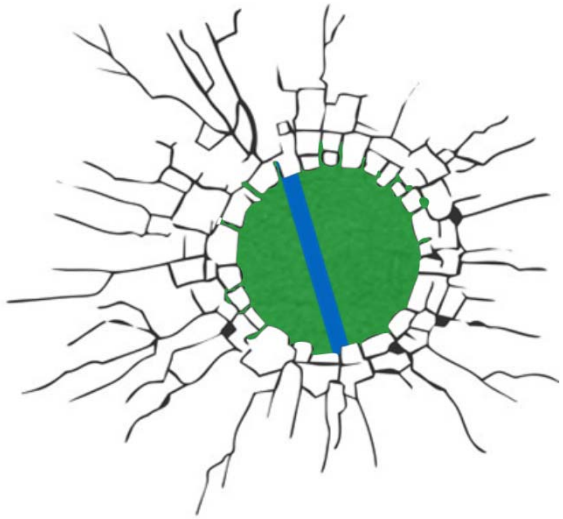
Optical Flow (Lucas-Kanade)

- Aperture Problem
  - In which direction is the line moving?



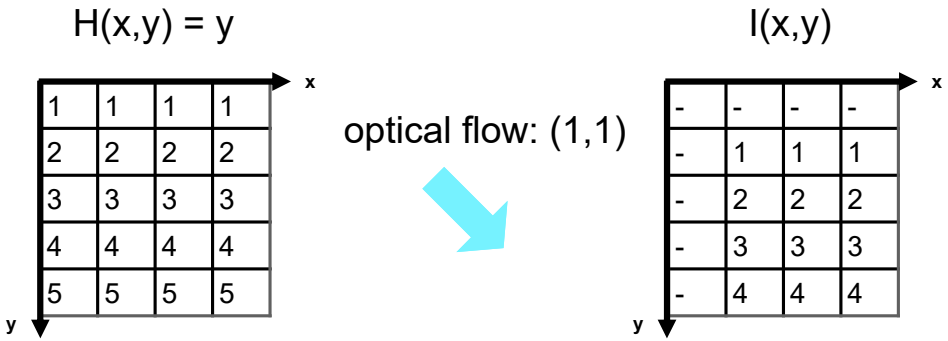
Optical Flow (Lucas-Kanade)

- Aperture Problem
  - In which direction is the line moving?



Optical Flow (Lucas-Kanade)

- Aperture Problem



$$I_x u + I_y v + I_t = 0$$

Compute gradients

$$I_x(3,3) = 0$$
$$I_y(3,3) = 1$$
$$I_t(3,3) = I(3,3) - H(3,3) = -1$$

Solution:

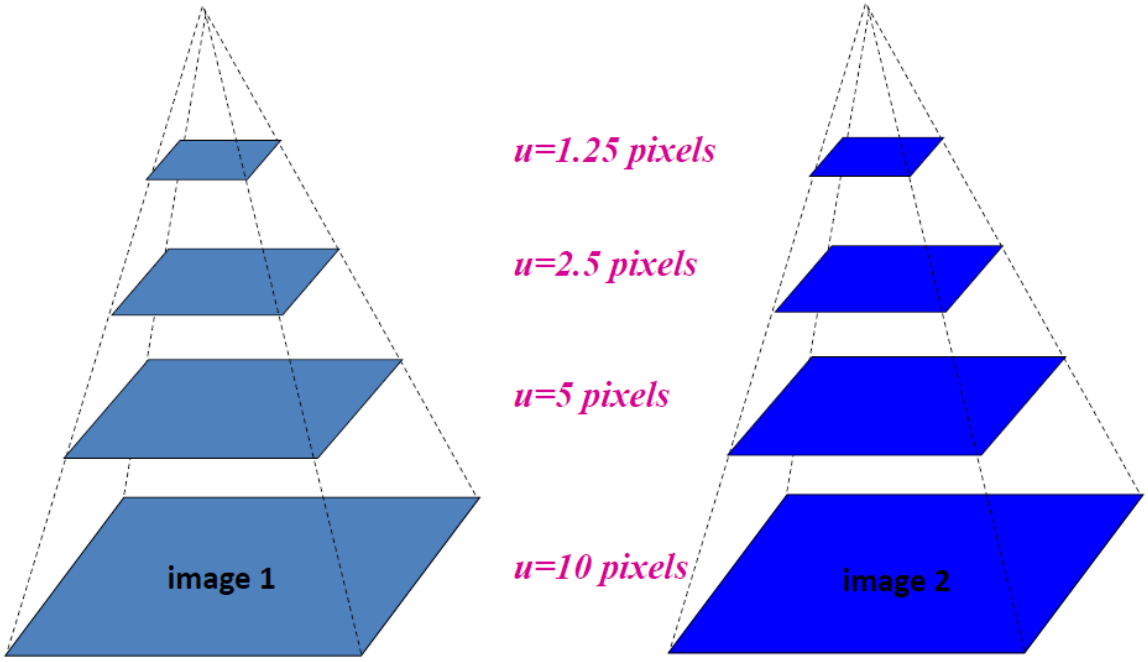
$$v = 1$$

We recover the v of the optical flow but not the u.

This is the aperture problem.

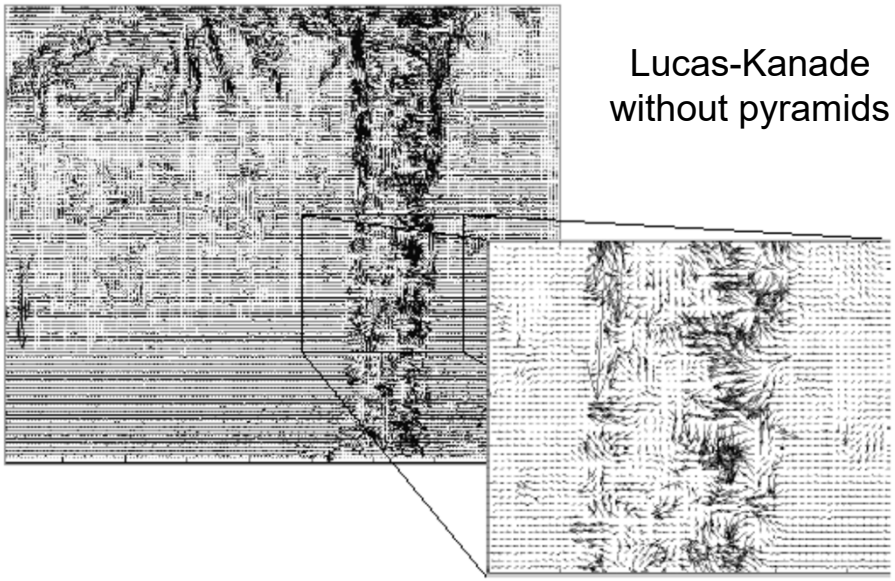
Optical Flow (Lucas-Kanade)

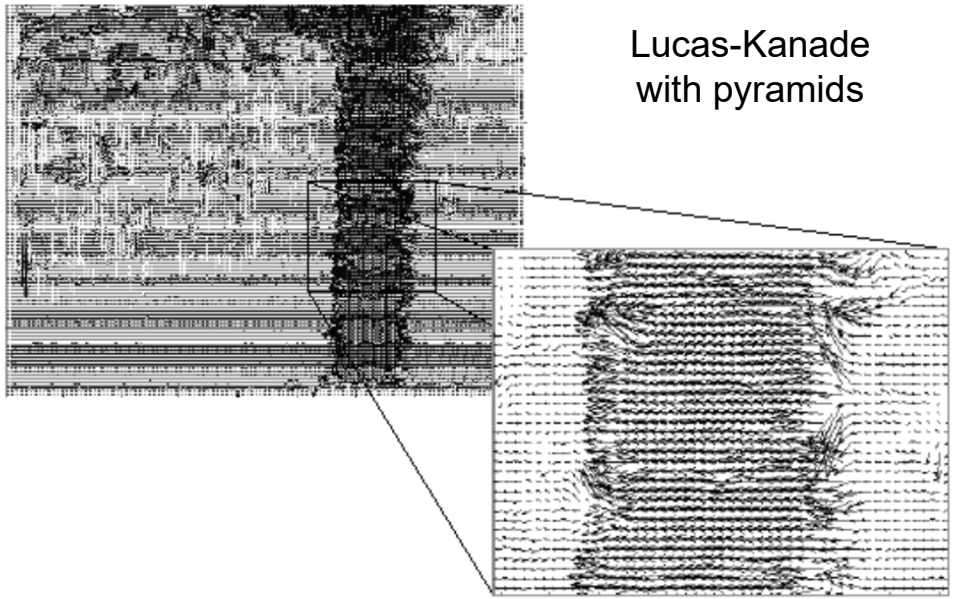
- Lucas-Kanade with pyramid
  - Coarse-to-fine optical flow estimation



Optical Flow (Lucas-Kanade)

- Lucas-Kanade with pyramids
  - Optical Flow Results





Optical Flow

http://vision.middlebury.edu/flow/
(Sometimes the link is not connected !!)

Lucas-Kanade Optical Flow (1981)

method of differences

'constant' flow
(flow is constant for all pixels)

local method
(sparse)

Horn-Schunck Optical Flow (1981)

brightness constancy
small motion

'smooth' flow
(flow can vary from pixel to pixel)

global method
(dense)

Optical Flow (Horn-Schunck)

- Key idea

Enforce **brightness constancy**

Enforce **smooth flow field**

Smoothness



Optical Flow (Horn-Schunck)

- The flow is formulated as a global energy function which is should be minimized:

α is a regularization constant.
Larger values of α lead to smoother flow

$$E = \iint \left[(I_x u + I_y v + I_t)^2 + \alpha^2 \left(\|\nabla u\|^2 + \|\nabla v\|^2 \right) \right] dx dy$$

The first part of the function is the brightness consistency

The second part is the smoothness constraint
It's trying to make sure that the changes between pixels are small

Optical Flow (Horn-Schunck)

- Energy function should be minimized

$$E = \iint \left[(I_x u + I_y v + I_t)^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2) \right] \mathrm{d}x\mathrm{d}y$$

This minimization can be solved by taking the derivative with respect to u and v, we get the following 2 equations:

Euler-Lagrange equations

$$I_x (I_x u + I_y v + I_t) - \alpha^2 \Delta u = 0$$
$$I_y (I_x u + I_y v + I_t) - \alpha^2 \Delta v = 0$$

Optical Flow (Horn-Schunck)

- Energy function should be minimized
  - By taking the derivative with respect to u and v, we get the following 2 equations:

$$I_x(I_x u + I_y v + I_t) - \alpha^2 \Delta u = 0$$
$$I_y(I_x u + I_y v + I_t) - \alpha^2 \Delta v = 0$$

  - Where $\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ is called the Lagrange operator. In practice, it is measured using:

$$\Delta u(x,y) = \bar{u}(x,y) - u(x,y)$$

  - where $\bar{u}(x,y)$ is the weighted average of u measured at (x,y).

Optical Flow (Horn-Schunck)

- Energy function should be minimized
  - To get:

$$(I_x^2 + \alpha^2)u + I_x I_y v = \alpha^2 \bar{u} - I_x I_t$$
$$I_x I_y u + (I_y^2 + \alpha^2)v = \alpha^2 \bar{v} - I_y I_t$$

  - Which is linear in u and v and can be solved analytically for each pixel individually

Cramers Rule

$$D = \begin{vmatrix} (I_x^2 + \alpha^2) & I_x I_y \\ I_x I_y & (I_y^2 + \alpha^2) \end{vmatrix} \quad D_u = \begin{vmatrix} -I_t I_x + \alpha^2 \bar{u} & I_x I_y \\ -I_t I_y + \alpha^2 \bar{v} & (I_y^2 + \alpha^2) \end{vmatrix} \quad D_v = \begin{vmatrix} (I_x^2 + \alpha^2) & -I_t I_x + \alpha^2 \bar{u} \\ I_x I_y & -I_t I_y + \alpha^2 \bar{v} \end{vmatrix}$$

Optical Flow (Horn-Schunck)

- Energy function should be minimized
  - Cramers Rule

$$D = (I_x^2 + \alpha^2)\left(I_y^2 + \alpha^2\right) - \left(I_x I_y\right)^2$$

$$D_u = (-I_t I_x + \alpha^2 \bar{u})\left(I_y^2 + \alpha^2\right) - \left(-I_t I_y + \alpha^2 \bar{v}\right)\left(I_x I_y\right)$$

$$D_v = \left(-I_t I_y + \alpha^2 \bar{v}\right)(I_x^2 + \alpha^2) - \left(-I_t I_y + \alpha^2 \bar{u}\right)\left(I_x I_y\right)$$

$$u = \frac{D_u}{D} = \frac{\bar{u}\left(I_y^2+I_x^2+\alpha^2\right)-I_x(I_x\bar{u}+I_y\bar{v}+I_t)}{\left(I_y^2+I_x^2+\alpha^2\right)}$$

$$= \bar{u} - I_x \frac{(I_x\bar{u}+I_y\bar{v}+I_t)}{\left(I_y^2+I_x^2+\alpha^2\right)}$$

$$v = \frac{D_v}{D} = \frac{\bar{v}\left(I_y^2+I_x^2+\alpha^2\right)-I_y(I_x\bar{u}+I_y\bar{v}+I_t)}{\left(I_y^2+I_x^2+\alpha^2\right)}$$

$$= \bar{v} - I_y \frac{(I_x\bar{u}+I_y\bar{v}+I_t)}{\left(I_y^2+I_x^2+\alpha^2\right)}$$

Optical Flow (Horn-Schunck)

- Energy function should be minimized
  - But since the solution depends on the neighboring values of the flow field, it must be repeated once the neighbors have been updated
  - So instead, we can iteratively solve for u and v using:

$$u^{k+1} = \bar{u}^k - \frac{I_x(I_x\bar{u}^k + I_y\bar{v}^k + I_t)}{\alpha^2 + I_x^2 + I_y^2}$$

$$v^{k+1} = \bar{v}^k - \frac{I_y(I_x\bar{u}^k + I_y\bar{v}^k + I_t)}{\alpha^2 + I_x^2 + I_y^2}$$

Optical Flow (Horn-Schunck)

- Algorithm

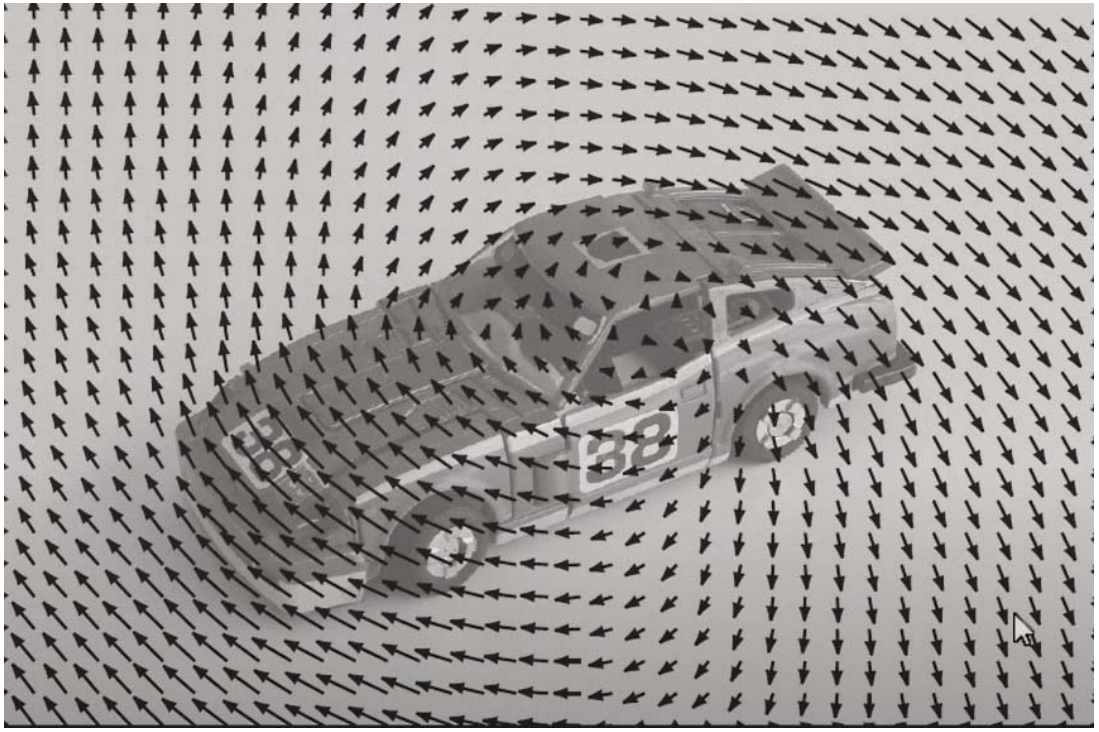
1. Initialize u and v as 0 matrices
2. Compute $I_x, I_y \; and \; I_t$ images using the masks
3. Update u^{k+1} , v^{k+1}
4. Repeat the above steps until some error measure is satisfied

Optical Flow (Horn-Schunck)

- Example







Visual Tracking

Visual Tracking

- Problem statement
  - Input: target
  - Objective: Estimate target state over time (space)
  - State:
    - Position
    - Appearance
    - Shape
    - Velocity
    - Affine transformation w.r.t. previous patch

Visual Tracking

- What are the challenges?
  - Variations due to geometric changes (pose, articulation, scale)
  - Variations due to photometric factors (illumination, appearance)
  - Occlusions
  - Non-linear motion
  - Very limited resolution, blurry (standard recognition might fail)
  - Similar objects in the scene

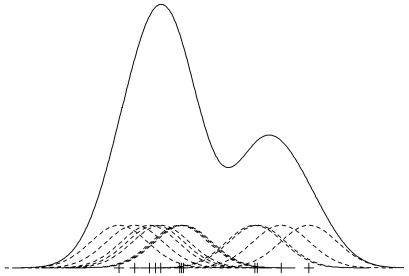
Visual Tracking

- Algorithms common issues
  - Track initiation & termination
  - Occlusion handling
  - Merging/switching
  - Drifting due to wrong update of the target model

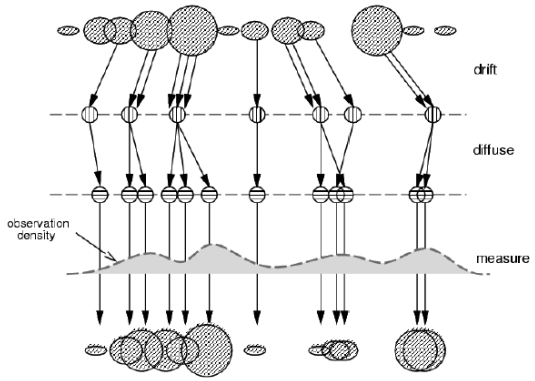
Tracking in Video



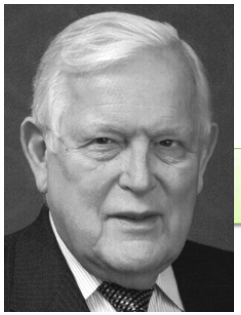
KLT

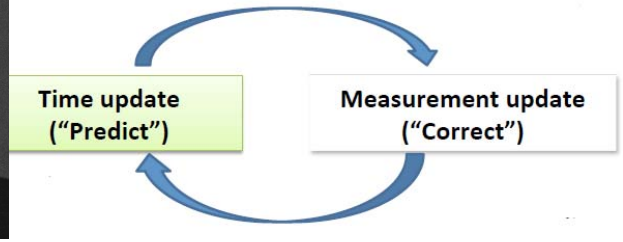


Mean shift

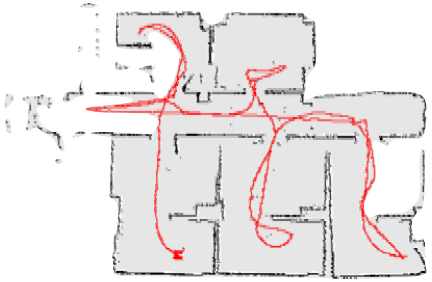


Particle Filtering





Kalman Filtering

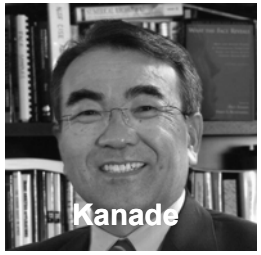


SLAM

Feature Tracking

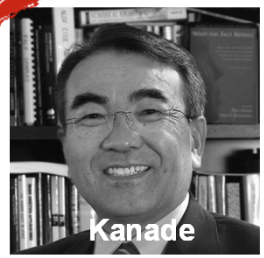
- Kanade-Lucas-Tomasi (KLT) Tracker





An Iterative Image Registration Technique with an Application to Stereo Vision.

1981





Detection and Tracking of Feature Points.

1991





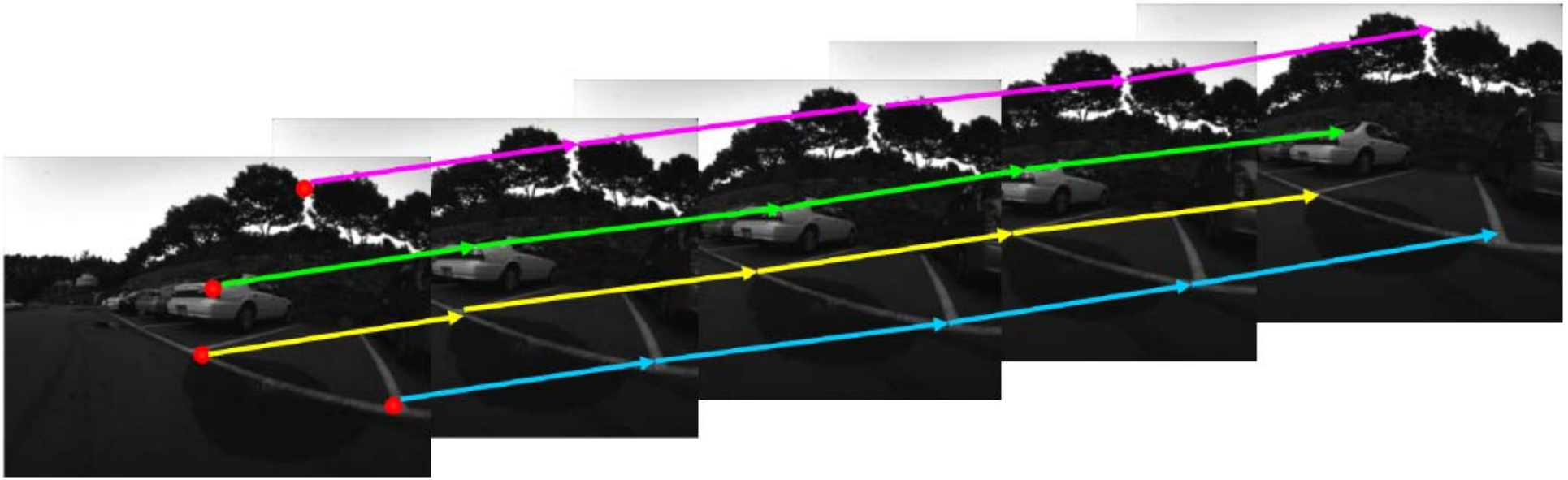
Good Features to Track.

1994

Visual Tracking

- Feature Tracking

Image sequence Feature point detection Feature point tracking



KLT algorithm

- What are good features for tracking?
  - Intuitively, we want to avoid smooth regions and edges
  - But is there a more is principled way to define good features?
  - Can be derived from the tracking algorithm

'A feature is good if it can be tracked well'

KLT algorithm

- Review of Optical Flow (Lucas-Kanade)

$$x = (A^\top A)^{-1} A^\top b$$

Where have you seen this before?

$$A^\top A = \begin{bmatrix} \sum_{p \in P} I_x I_x & \sum_{p \in P} I_x I_y \\ \sum_{p \in P} I_y I_x & \sum_{p \in P} I_y I_y \end{bmatrix}$$

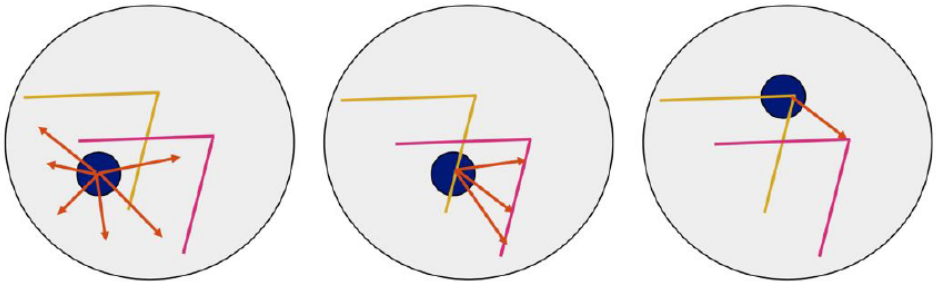
Harris Corner Detector!

- A^TA should be invertible
- A^TA should not be too small
  - λ_1 and λ_2 should not be too small
- A^TA should be well conditioned
  - λ_1/λ_2 should not be too large (λ_1=larger eigenvalue)

KLT algorithm

- Implications
  - Corners are when λ_1, λ_2 are big; this is also when Lucas-Kanade optical flow works best
  - Corners are regions with two different directions of gradient (at least)
  - Corners are good places to compute flow!

What happens when you have no 'corners'?



$\lambda_1 \simeq 0, \lambda_2 \simeq 0$ λ_1 large, $\lambda_2 \simeq 0$ λ_1, λ_2 : large

KLT algorithm

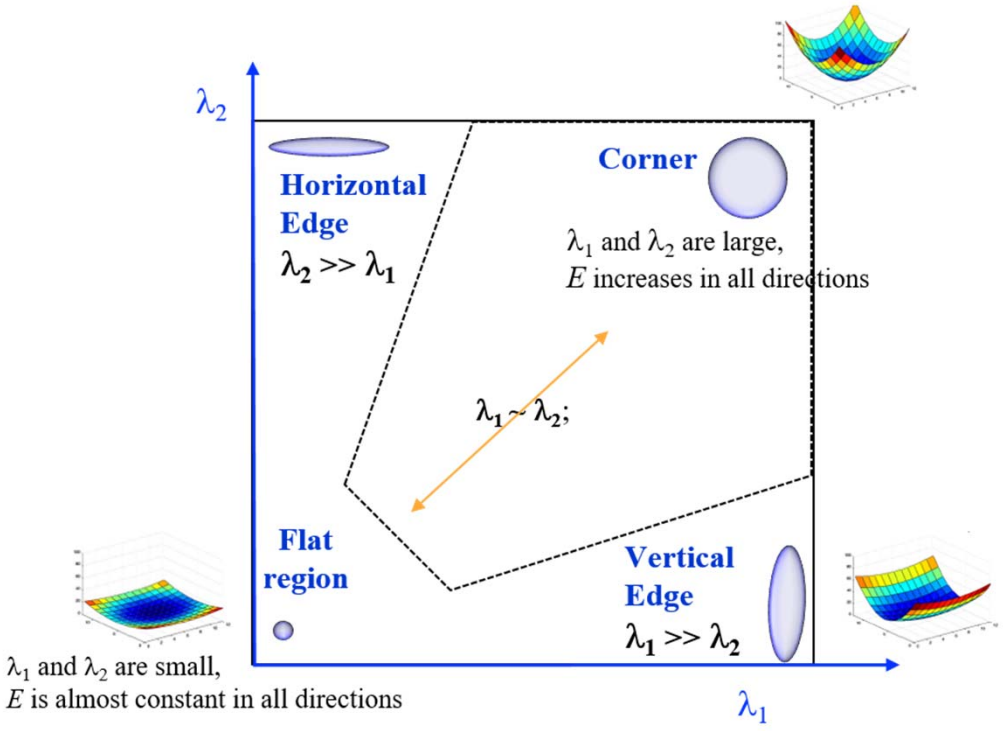
- How are the eigenvalues related to image content?

$$A^\top A = \begin{bmatrix} \sum_{p \in P} I_x I_x & \sum_{p \in P} I_x I_y \\ \sum_{p \in P} I_y I_x & \sum_{p \in P} I_y I_y \end{bmatrix}$$

interpreting eigenvalues

$$\min(\lambda_1, \lambda_2) > \lambda$$

'big Eigenvalues means good for tracking'



KLT algorithm

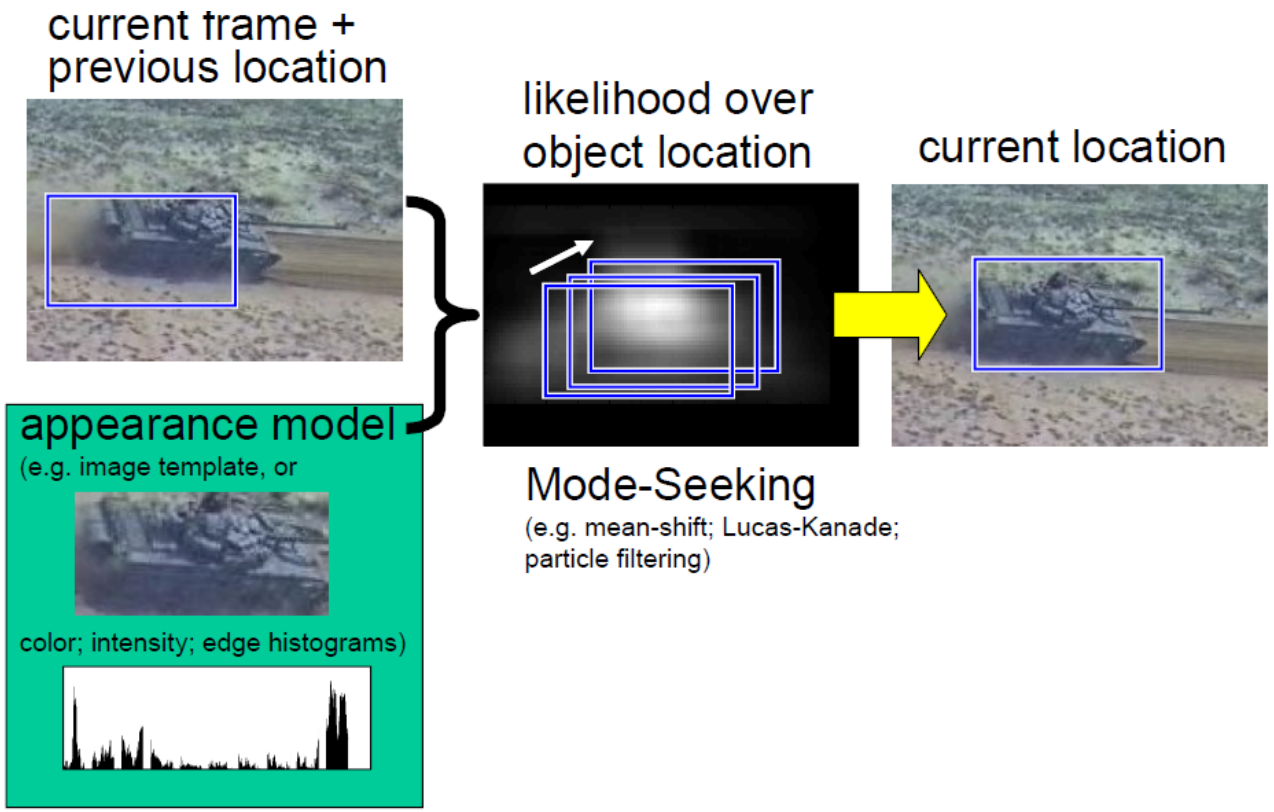
https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_video/py_lucas_kanade/py_lucas_kanade.html

- Steps of algorithm

1. Find corners satisfying $\min(\lambda_1, \lambda_2) > \lambda$
2. For each corner compute displacement to next frame using the Lucas-Kanade method
3. Store displacement of each corner, update corner position
4. (optional) Add more corner points every M frames using 1
5. Repeat 2 to 3 (4)
6. Returns long trajectories for each corner point

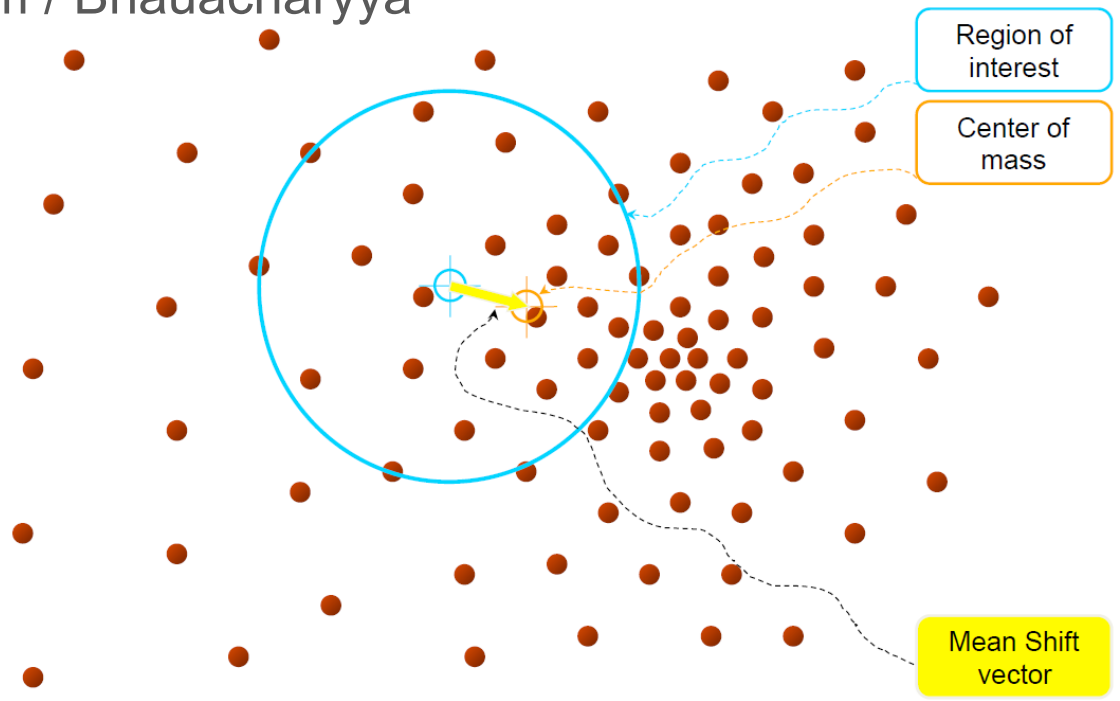
Mean Shift Tracking

- Appearance-Based Tracking



Mean Shift Tracking

- Non-parametric feature space
- Locate the maxima of a density function
- Color histogram / Bhauacharyya



Mean Shift Tracking

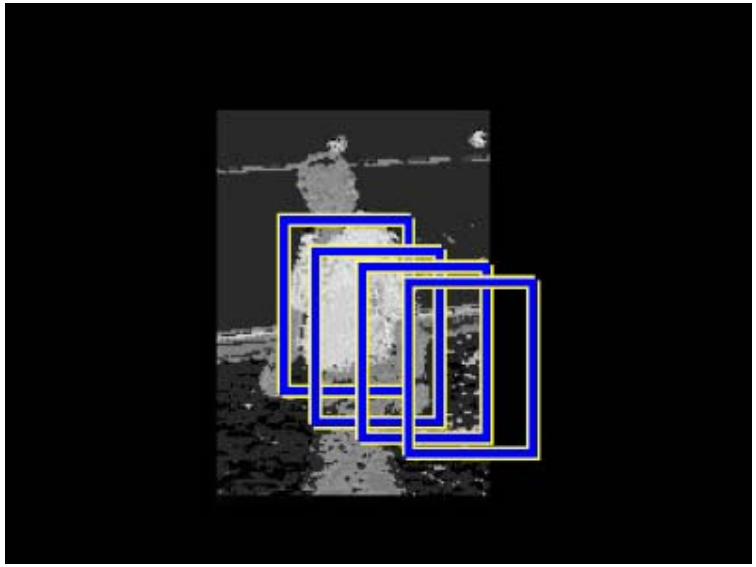
- Two approaches
  - Create a color “likelihood” image, with pixels weighted by similarity to the desired color (best for unicolored objects)
  - Represent color distribution with a histogram. Use mean-shift to find region that has most similar distribution of colors.

Computation of likelihood can be based on
- color
- texture
- shape (boundary)
- predicted location



Mean Shift Tracking

- Let pixels form a uniform grid of data points, each with a weight (pixel value) proportional to the “likelihood” that the pixel is on the object we want to track
- Perform standard mean-shift algorithm using this weighted set of points

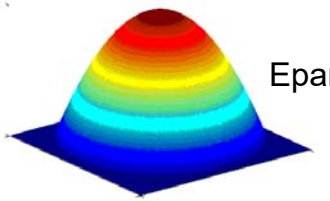


$$\Delta x = \frac{\sum w(x)(x - x_{old})K(x - x_{old})}{\sum w(x)K(x - x_{old})}$$

Mean Shift

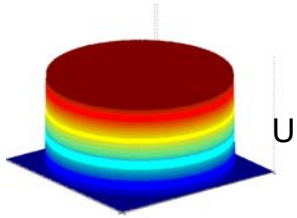
- Kernel function

These are all radially symmetric kernels



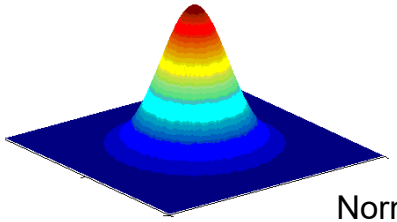
Epanechnikov kernel

$$K(\boldsymbol{x}, \boldsymbol{x}') = \begin{cases} c(1 - \|\boldsymbol{x} - \boldsymbol{x}'\|^2) & \|\boldsymbol{x} - \boldsymbol{x}'\|^2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$



Uniform kernel

$$K(\boldsymbol{x}, \boldsymbol{x}') = \begin{cases} c & \|\boldsymbol{x} - \boldsymbol{x}'\|^2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

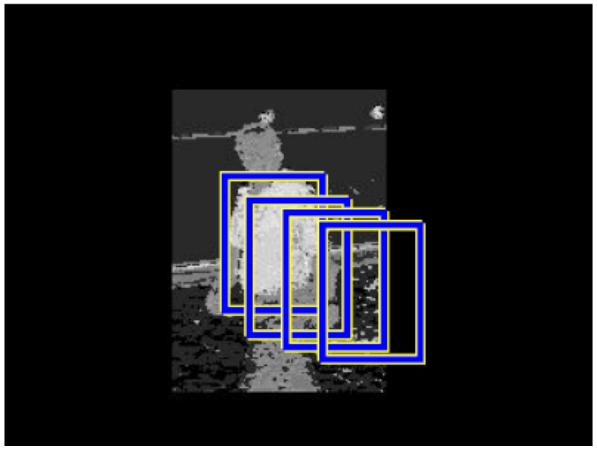


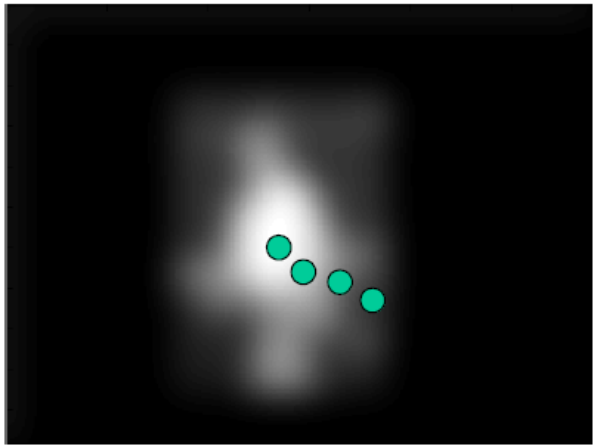
Normal kernel

$$K(\boldsymbol{x}, \boldsymbol{x}') = c \exp\left(\frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}'\|^2\right)$$

Mean Shift Tracking

- Nice Property
  - Running mean-shift with kernel K on weight image w is equivalent to performing gradient ascent in a (virtual) image formed by convolving w with some “shadow” kernel
  - Note: mode we are looking for is mode of location (x,y) likelihood, NOT mode of the color distribution!





Mean Shift Tracking

Object tracking with Mean-shift – OpenCV 3.4 with python 3
https://www.youtube.com/watch?v=EDT0vHsMy34&t=216s

https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_video/py_meanshift/py_meanshift.html

- Algorithm
  1. Initialize location y_0
  2. Compute $H_{model} = \{q_m\}_{m=1,..n}$, $\sum_1^n q_m = 1$
  3. Make histogram projection of consecutive frame(image) $I(x)$

     with $w(x) = \sqrt{\frac{H_{model}(I(x))}{H_{I(x)}(I(x))}}$
  4. Calculate mean-shift localization $y_{new} = \frac{\sum w(x_i)x_i K(x,x_i)}{\sum w(x_i)K(x,x_i)}$

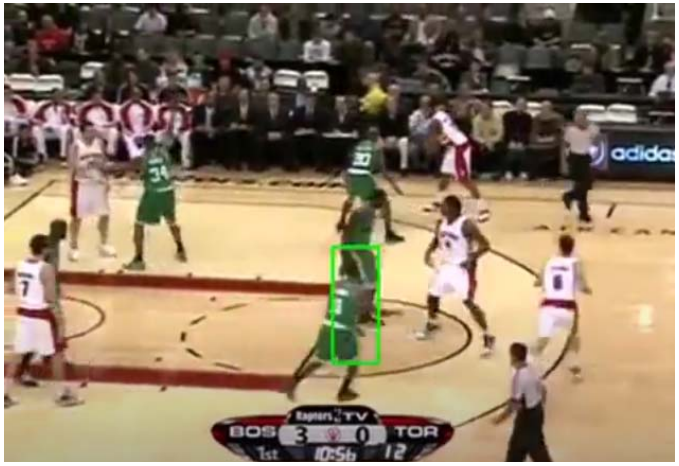
     until $\|y_{new} - y_{old}\| < \varepsilon$

Mean Shift Tracking

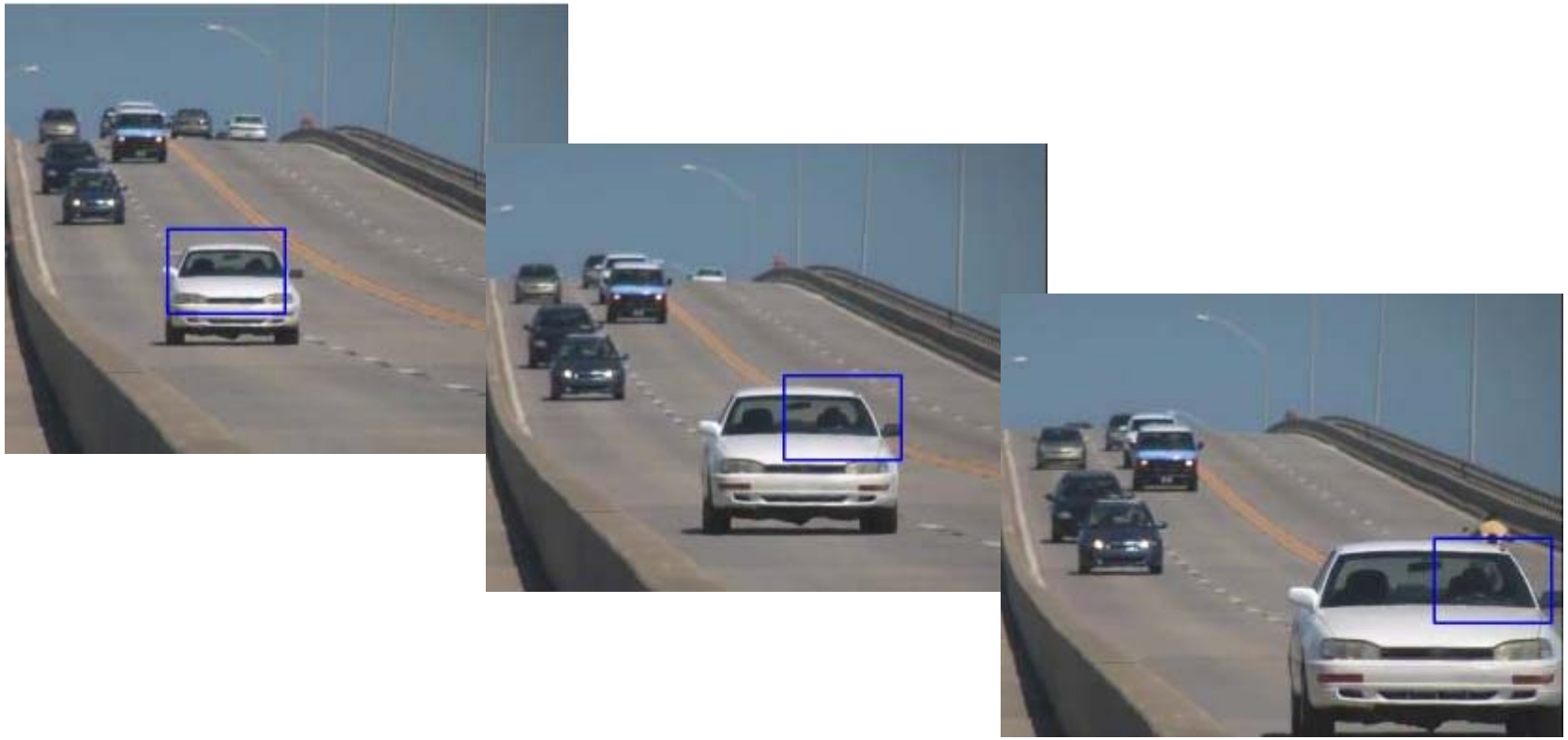
- Result 1





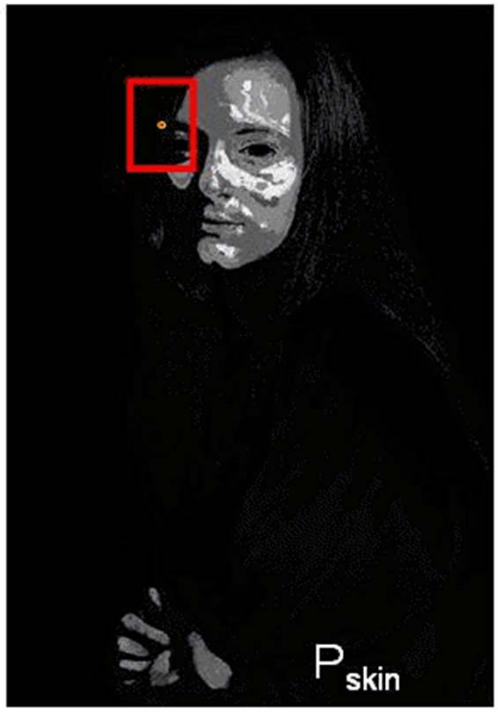


- Result 2



CAM Shift Tracking

- CAMshift (Continuously Adaptive Meanshift)
  - Mean shift window always has the same size
  - It need to adapt the window size with size and rotation of the target
  - The algorithm is almost same. But, it updates the size of the window as, $s = 2 \times \sqrt{\frac{M_{00}}{256}}$



Mean shift window initialization

CAM Shift Tracking

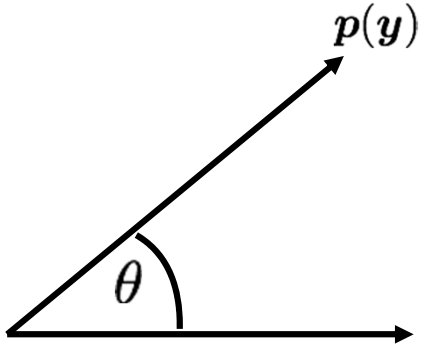
- Similarity between the target and candidate
  - Using Bhattacharyya distance

Distance function $$d(\boldsymbol{y}) = \sqrt{1 - \rho[\boldsymbol{p}(\boldsymbol{y}), \boldsymbol{q}]}$$

Bhattacharyya Coefficient $$\rho(y) \equiv \rho[\boldsymbol{p}(\boldsymbol{y}), \boldsymbol{q}] = \sum_m \sqrt{p_m(\boldsymbol{y}) q_u}$$

Just the Cosine distance between two unit vectors

$$\rho(\boldsymbol{y}) = \cos\theta_{\boldsymbol{y}} = \frac{\boldsymbol{p}(\boldsymbol{y})^\top \boldsymbol{q}}{\|\boldsymbol{p}\|\|\boldsymbol{q}\|} = \sum_m \sqrt{p_m(\boldsymbol{y}) q_m}$$



CAM Shift Tracking

Object tracking with Camshift – OpenCV 3.4 with python 3
https://www.youtube.com/watch?v=a9KZjQ4e6IA

- Result



Any Questions?

Thanks!